FULL STACK DEVELOPMENT

Nexus CRM

Complete Learning Guide and Implementation Reference

Scope: Semester MVP — a functional CRM from lead capture to closed revenue

Stack: React 19 · Vite · Tailwind CSS v4 · Express · SQLite · Prisma

Generated: 2026-10-14

Sources: README.md and the 14 documents in `docs/`

Contents

PART I — FOUNDATIONS

1. Project Master Overview

2. Requirements

3. Architecture

PART II — DATA LAYER

4. Database Guide

5. API Reference

PART III — FRONTEND

6. JavaScript Guide

7. React Guide

8. Frontend Guide

PART IV — OPERATIONS

9. Setup and Run

10. Testing Report

11. Known Limitations

12. Future Roadmap

PART V — PRESENTATION

13. Final Presentation Outline

14. Viva Question Bank

README

A full-stack Customer Relationship Management web application built for a university semester project and designed to evolve into a production-grade CRM product.

Features

- **Dashboard** — KPI cards, charts (Recharts), lead acquisition trends, revenue overview
- **Contacts** — Full CRUD, search, filter, lifecycle stages, duplicate email prevention
- **Leads** — Lead management with MQL/SQL/Deal status, qualification workflow, contact association
- **Deals & Pipeline** — Kanban-style pipeline view, stage management, deal creation and tracking
- **Pipelines** — Create/edit pipelines, add/rename/reorder stages, safe deletion
- **Lead Sources** — Simulated Google Ads, Meta Ads, Website Form, and Inbound Call integrations
- **Activities** — Call, email, meeting, note, and task tracking with due dates
- **Reports** — Analytics charts with CSV export, live data from application database

Tech Stack

Layer	Technology
Frontend	React 19, Vite 8, Tailwind CSS 4, Recharts, Lucide React
Backend	Node.js, Express.js
Database	SQLite, Prisma ORM
State	React Hooks (useState, useEffect)
Routing	React Router DOM 7

Quick Start

```
npm install

echo "DATABASE_URL=\"file:./server/prisma/dev.db\"" > .env
npx prisma db push --schema server/prisma/schema.prisma
node server/prisma/seed.js

npm run server

npm run dev
```

Visit `http://localhost:5173` to use the application.

Project Structure

```
crm-platform/
  src/
    components/
      common/       - NotFound page
      dashboard/    - Dashboard with KPI cards and charts
      contacts/     - Contacts CRUD with search/filter
      leads/        - Leads management with qualification
      deals/        - Kanban pipeline and deal CRUD
      pipelines/    - Pipeline and stage management
      sources/      - Simulated lead source integrations
      activities/   - Activity and follow-up tracking
      reports/      - Analytics and CSV export
      ui/           - Reusable UI components (Button, Card, Badge, Modal, Input)
      layout/       - AppShell, Sidebar, Header
    services/       - API client (api.js)
    utils/          - Constants and helpers
    pages/          - (routed via AppShell)
    App.jsx         - Route configuration
    main.jsx        - Entry point with Router
  server/
    server.js       - Express app entry
    routes/         - REST API route handlers
    controllers/    - Business logic and Prisma operations
    prisma/
      schema.prisma - Database schema
      seed.js       - Sample data
    .env            - DATABASE_URL
  docs/             - Documentation and learning guides
```

Demo Walkthrough

1. Open the Dashboard → see seeded KPIs and charts
2. Go to **Lead Sources** → click **Simulate Google Ads Lead** → see success toast
3. Go to **Leads** → find the new lead with MQL badge
4. Click the **Qualify** button → select SQL/Deal type, enter contact details
5. Go to **Deals** → create a deal for the lead → move through pipeline stages
6. Go to **Dashboard** → see updated metrics and charts

Scripts

Command	Description
`npm run dev`	Start Vite dev server
`npm run server`	Start Express backend
`npm run dev:all`	Start both concurrently
`npm run build`	Production build
`npm run db:push`	Push schema to database
`npm run db:seed`	Seed with sample data
`npm run db:studio`	Open Prisma Studio
`npm run lint`	Run ESLint

Documentation

See `docs/` for:

- Architecture, database schema, and API reference
- Frontend and JavaScript learning guides
- Testing report and known limitations
- Viva question bank (100+ questions)
- Complete learning guide with code explanations

License

University semester project — Nexus CRM.

PART I — FOUNDATIONS

Project Master Overview

1. Project Overview

Name: Nexus CRM **Type:** Full-stack CRM web application **Purpose:** Semester 1 project + personal mega project **Timeline:** ~14 days, 3-4 hours/day

2. Product Workflow

```
Lead Sources → New Lead / MQL → Contact Association → Qualification → SQL
    → Deal Creation → Sales Pipeline → Closed Won / Closed Lost
    → Dashboard & Analytics
```

Lead Sources

- Google Ads (simulated)
- Meta Ads (simulated)
- Website Forms (simulated)
- Inbound Calls (simulated)
- Referrals (manual entry)
- Manual Entry

Lead Lifecycle

Status	Type	Description
New	MQL	Fresh lead, no contact yet
Attempting Contact	MQL	Trying to reach
Connected	MQL	Contact established
Qualified	SQL	Ready for sales
Converted	Deal	Became an opportunity

3. Technology Stack

Layer	Technology	Version
Frontend Framework	React	19.2.8
Build Tool	Vite	8.3.0
Styling	Tailwind CSS	4.3.3
Charts	Recharts	3.10.1
Icons	Lucide React	1.51.0
Routing	React Router DOM	7.18.4
Backend	Express.js	5.3.0
Database	SQLite + Prisma	6.8.0
Notifications	React Hot Toast	—

4. Application Modules

A. Application Shell

- Responsive sidebar with navigation
- Top header with search, notifications, user profile
- Breadcrumbs
- React Router navigation
- Collapsible sidebar

B. Dashboard (8 KPI Cards + 4 Charts)

- Total Contacts, Total Leads, MQLs, SQLs
- Open Deals, Open Pipeline Value, Closed Won Revenue, Conversion Rate
- Lead Acquisition Over Time (Line Chart)
- Leads by Source (Pie Chart)
- Lead Status Distribution (Bar Chart)
- Revenue Overview (Pipeline vs Won vs Lost)
- Recent Leads, Recent Activities

C. Contacts

- Full CRUD operations
- Search, sort, lifecycle stage filter
- Duplicate email prevention
- Contact detail view with associated leads/deals/activities
- Form validation

D. Leads

- List with search, status/source/type filters
- MQL/SQL/Deal type badges
- Contact association (create or link existing)
- Qualification workflow (MQL → SQL → Deal)
- Status progression
- Activity history

E. Deals & Pipeline

- Kanban board with pipeline stages
- Move deals forward/backward between stages
- Deal amount, probability, expected close date
- Create/edit/delete deals
- Pipeline selector filter
- Stage totals and deal counts

F. Customizable Pipelines

- Create/rename pipelines
- Add/rename/delete stages
- Configure stage colors
- Mark stages as Closed Won/Closed Lost
- Safe deletion (reassign deals first)

G. Lead Sources

- 4 simulated source cards (Google Ads, Meta Ads, Website Form, Inbound Call)
- Each creates realistic records with contact association
- Clear simulation labels
- Success feedback with toast notifications

H. Activities & Follow-ups

- Create calls, emails, meetings, notes, tasks
- Due dates and priority levels
- Mark activities complete
- Filter by type

I. Reports

- Leads by source, status, over time
- Deals by stage

- Pipeline value
- Won/lost distribution
- CSV export

5. Database Schema Summary

```
User (id, email, name, role)
  └→ Lead (createdById)
  └→ Deal (createdById)
  └→ Activity (createdById)

Contact (id, firstName, lastName, email, phone, company, jobTitle, location, leadSource, lifecycle
  ├→ Lead (contactId) – optional
  ├→ Deal (contactId)
  └→ Activity (contactId) – optional

Lead (id, leadNumber, status, type, source, company, notes, qualificationNotes, contactId)
  ├→ Contact (contactId)
  ├→ Activity (leadId) – optional
  └→ User (createdById)

Pipeline (id, name, description, isDefault)
  └→ Stage (pipelineId)
  └→ Deal (pipelineId)

Stage (id, name, position, color, isClosedWon, isClosedLost)
  └→ Deal (stageId)

Deal (id, dealName, amount, probability, expectedCloseDate, actualCloseDate, status, lostReason, r
  ├→ Contact (contactId)
  ├→ Stage (stageId)
  ├→ Pipeline (pipelineId)
  ├→ User (createdById)
  └→ Activity (dealId) – optional

Activity (id, type, subject, description, dueDate, completed, priority)
  ├→ Contact (contactId) – optional
  ├→ Lead (leadId) – optional
  ├→ Deal (dealId) – optional
  └→ User (createdById)
```

6. API Routes

Contacts

- `GET /api/contacts` — List with search/sort/filter
- `GET /api/contacts/:id` — Get one
- `POST /api/contacts` — Create
- `PUT /api/contacts/:id` — Update

- `DELETE /api/contacts/:id` — Delete

Leads

- `GET /api/leads` — List with filters
- `GET /api/leads/:id` — Get one
- `POST /api/leads` — Create
- `PUT /api/leads/:id` — Update
- `DELETE /api/leads/:id` — Delete
- `POST /api/leads/:id/qualify` — Qualify (MQL→SQL→Deal)
- `POST /api/leads/:id/convert` — Convert to deal

Deals

- `GET /api/deals` — List
- `GET /api/deals/:id` — Get one
- `POST /api/deals` — Create
- `PUT /api/deals/:id` — Update
- `DELETE /api/deals/:id` — Delete
- `PATCH /api/deals/:id/stage` — Move to stage

Pipelines

- `GET /api/pipelines` — List all
- `GET /api/pipelines/:id` — Get one
- `POST /api/pipelines` — Create
- `PUT /api/pipelines/:id` — Update
- `DELETE /api/pipelines/:id` — Delete
- `POST /api/pipelines/:id/stages` — Add stage
- `PUT /api/pipelines/stages/:id` — Update stage
- `DELETE /api/pipelines/stages/:id` — Delete stage

Dashboard

- `GET /api/dashboard` — All dashboard metrics

Activities

- `GET /api/activities` — List with filters
- `GET /api/activities/:id` — Get one
- `POST /api/activities` — Create
- `PUT /api/activities/:id` — Update
- `DELETE /api/activities/:id` — Delete

Sources

- `POST /api/sources/google-ads` — Simulate Google Ads lead
- `POST /api/sources/meta-ads` — Simulate Meta Ads lead
- `POST /api/sources/website-form` — Simulate website lead
- `POST /api/sources/inbound-call` — Simulate inbound call lead

Reports

- `GET /api/reports` — Aggregated analytics data

Health

- `GET /api/health` — Server health check

Requirements

Functional Requirements

FR-1: Dashboard

- Display 8 KPI cards with live data
- Lead acquisition over time chart
- Leads by source pie chart
- Lead status distribution bar chart
- Revenue overview (pipeline/won/lost)
- Recent leads and activities lists

FR-2: Contacts Management

- Create, read, update, delete contacts
- Search by name, email, company
- Filter by lifecycle stage
- Sort by any field
- Duplicate email prevention
- Contact detail view
- Associated leads, deals, and activities

FR-3: Leads Management

- Create, read, update, delete leads
- Search by lead number, company, notes
- Filter by status, source, type
- Lead types: MQL, SQL, Deal
- Lead statuses: New, Attempting, Connected, Qualified, Disqualified, Converted
- Contact association (existing or new)
- Qualification workflow
- Notes and follow-up dates

FR-4: Deals & Pipeline

- Create, read, update, delete deals
- Kanban board by pipeline stages
- Move deals between stages
- Deal name, amount, probability, close date
- Filter by pipeline

- Stage totals and counts

FR-5: Pipeline Customization

- Create/rename pipelines
- Add/rename/reorder stages
- Configure stage color
- Mark stage as Closed Won/Closed Lost
- Safe stage deletion

FR-6: Lead Sources

- Simulate 4 lead source types
- Create contact + lead automatically
- Preserve source and timestamp
- Success feedback

FR-7: Activities

- Create activities (call, email, meeting, note, task)
- Due dates, priority, completion status
- Associate with contact/lead/deal
- Filter by type
- Mark complete

FR-8: Reports

- Leads by source, status, over time
- Deals by stage
- Pipeline value
- Won/lost distribution
- CSV export

FR-9: Navigation

- Responsive sidebar
- Active page indication
- Breadcrumbs
- Header with search

Non-Functional Requirements

NFR-1: Performance

- Dashboard loads in under 2 seconds
- List pages support pagination-ready structure
- API responses under 500ms

NFR-2: Persistence

- All data survives page reloads
- SQLite database with Prisma ORM
- Seed data for demonstration

NFR-3: Design

- Professional SaaS appearance
- Consistent design system
- Responsive (desktop, tablet, mobile)
- Proper loading/error/empty states

NFR-4: Code Quality

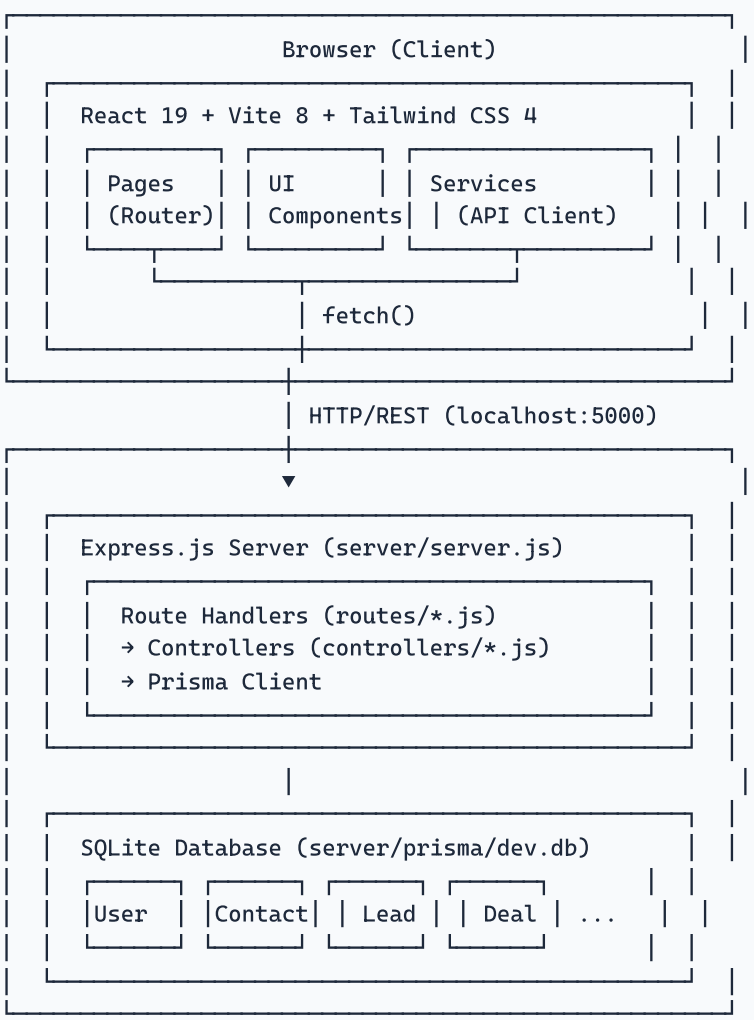
- Readable JavaScript ES6+
- Component reusability
- Separation of concerns
- Consistent naming conventions

Out of Scope (Semester)

- Real ad/telephony integrations
- Authentication and authorization
- Email sending
- Real-time notifications
- File uploads
- Multi-tenant support

Architecture

System Architecture



Frontend Architecture

Component Hierarchy

```
App
├── BrowserRouter
│   └── Routes
│       ├── / → AppShell + Dashboard
│       ├── /contacts → AppShell + Contacts
│       ├── /leads → AppShell + Leads
│       ├── /deals → AppShell + Deals
│       ├── /pipelines → AppShell + Pipelines
│       ├── /sources → AppShell + Sources
│       ├── /activities → AppShell + Activities
│       ├── /reports → AppShell + Reports
│       └── * → AppShell + NotFound

AppShell
├── Sidebar (navigation)
├── Header (breadcrumbs, search, profile)
└── <main> (page content)
```

Data Flow

1. User interacts with a component → triggers state update
2. Component calls API function from `src/services/api.js`
3. API function makes `fetch()` request to Express backend
4. Backend route handler calls controller
5. Controller uses Prisma Client to query/update database
6. Response flows back through the same chain
7. Component updates its state → re-renders with new data

State Management

- Local component state with `useState`
- `useEffect` for data loading
- No global state management (Redux/Zustand) for simplicity
- Prop drilling for component communication

Backend Architecture

Express Route Pattern

```
routes/*.js → Express Router
  ├── GET / → List all (with query params for filtering)
  ├── GET /:id → Get one
  ├── POST / → Create
  ├── PUT /:id → Update
  └── DELETE /:id → Delete
```

Error Handling

- Try/catch in every async route handler
- HTTP status codes: 200 (OK), 201 (Created), 400 (Bad Request), 404 (Not Found), 500 (Server Error)
- Consistent error response format: `{ error: "message" }`

Database Architecture

Prisma Workflow

1. Define schema in `server/prisma/schema.prisma`
2. Run `npx prisma db push` to create/update SQLite database
3. Prisma auto-generates `@prisma/client`
4. Controllers use Prisma Client for type-safe queries

Relationships

- Contact 1→N Lead, Deal, Activity
- User 1→N Lead, Deal, Activity
- Pipeline 1→N Stage, Deal
- Stage 1→N Deal
- Deal 1→N Activity

PART II — DATA LAYER

Database Guide

Schema Overview

The database uses SQLite with Prisma ORM. All tables and relationships are defined in `server/prisma/schema.prisma`.

Entity Relationship Diagram

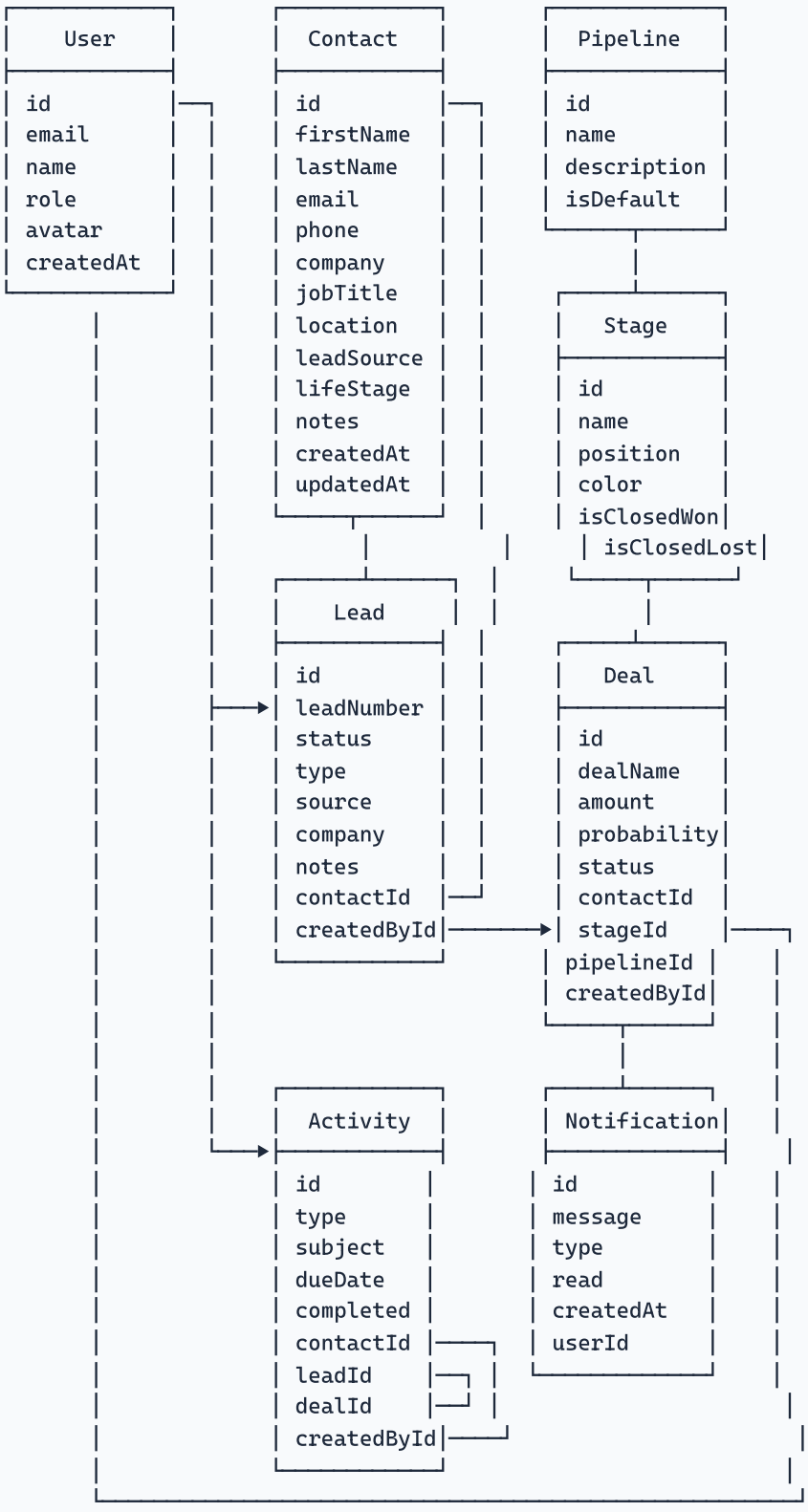


Table Descriptions

User

Stores application users. Currently minimal (admin user for seeding).

Field	Type	Notes
id	Int	Primary key
email	String	Unique
name	String	Full name
role	String	"user" or "admin"
avatar	String?	Profile image URL
createdAt	DateTime	Auto-set

Contact

Central entity for people/organizations. Connected to leads, deals, and activities.

Field	Type	Notes
id	Int	Primary key
firstName, lastName	String	Required
email	String	Unique, duplicate prevention
phone	String?	Optional
company	String?	Organization name
jobTitle	String?	Role in organization
location	String?	City/region
leadSource	String	Source of lead
lifecycleStage	String	lead/mql/sql/customer
notes	String?	Free text
createdAt	DateTime	Auto-set
updatedAt	DateTime	Auto-updated
lastActivityAt	DateTime	Updated on activity

Lead

Tracks potential customers through qualification stages.

Field	Type	Notes
id	Int	Primary key
leadNumber	String	Unique, e.g., "LEAD-ABC123"
status	String	new/attempting/connected/qualified/disqualified/converted
type	String	mql/sql/deal
source	String	google_ads/meta_ads/website/inbound_call/referral/manual
company	String?	Company name
companySize	String?	e.g., "1-10", "11-50"
industry	String?	Industry vertical
website	String?	Company website
notes	String?	Lead notes
qualificationNotes	String?	Sales qualification notes
nextFollowUp	DateTime?	Scheduled follow-up
expectedValue	Float?	Potential deal value
contactId	Int?	FK → Contact
createdById	Int	FK → User

Pipeline

Represents a sales pipeline (e.g., "Default Sales Pipeline").

Field	Type	Notes
id	Int	Primary key
name	String	Pipeline name
description	String?	Optional description
isDefault	Boolean	Default pipeline flag
createdAt	DateTime	Auto-set

Stage

Individual stages within a pipeline.

Field	Type	Notes
id	Int	Primary key
name	String	Stage name
position	Int	Order in pipeline
color	String	Hex color for display
isClosedWon	Boolean	Marks won stage
isClosedLost	Boolean	Marks lost stage
pipelineId	Int	FK → Pipeline

Deal

Sales opportunities tied to contacts and pipeline stages.

Field	Type	Notes
id	Int	Primary key
dealName	String	Deal name
amount	Float	Deal value
probability	Int	0-100 win probability
expectedCloseDate	DateTime?	Target close
actualCloseDate	DateTime?	When actually closed
status	String	open/won/lost
lostReason	String?	If status=lost
notes	String?	Deal notes
contactId	Int	FK → Contact
stageId	Int	FK → Stage
pipelineId	Int	FK → Pipeline
createdById	Int	FK → User

Activity

Tracks all interactions and tasks.

Field	Type	Notes
id	Int	Primary key
type	String	call/email/meeting/note/task
subject	String	Activity title
description	String?	Details
dueDate	DateTime?	Scheduled time
completed	Boolean	Completion flag
priority	String	low/medium/high
contactId	Int?	Optional FK → Contact
leadId	Int?	Optional FK → Lead
dealId	Int?	Optional FK → Deal
createdById	Int	FK → User

Notification

User notifications (stored, not actively rendered in frontend).

Field	Type	Notes
id	Int	Primary key
message	String	Notification text
type	String	info/success/warning/error
read	Boolean	Read status
createdAt	DateTime	Timestamp
userId	Int	FK → User

Migrations

Schema changes are applied via:

```
npx prisma db push --schema server/prisma/schema.prisma
```

This is development-mode migration (no migration files). For production, use:

```
npx prisma migrate dev --name description
```

Seeding

```
npm run db:seed
node server/prisma/seed.js
```

Creates: 1 admin user, 8 contacts, 8 leads, 7 deals, 1 pipeline with 7 stages, 7 activities.

API Reference

Base URL: `http://localhost:5000/api`

All endpoints return JSON. Error responses follow the format: `{ "error": "description" }`

Health

- `GET /health` → `{ "status": "ok" }`

Contacts

List Contacts

`GET /contacts?search=&sortBy=createdAt&sortOrder=desc&lifecycleStage=`

Query params:

- `search` (string) — Search in name, email, company
- `sortBy` (string) — Field to sort by
- `sortOrder` (string) — "asc" or "desc"
- `lifecycleStage` (string) — Filter by stage

Response: `Contact[]` (includes leads, deals, activities)

Get Contact

`GET /contacts/:id`

Response: `Contact | null` (404 if not found)

Create Contact

`POST /contacts`

Body:

```
{
  "firstName": "Rajesh",
  "lastName": "Kumar",
  "email": "rajesh@example.com",
  "phone": "+91 9876543210",
  "company": "TechVista",
  "jobTitle": "CTO",
  "location": "Bangalore",
  "leadSource": "google_ads",
  "lifecycleStage": "lead",
  "notes": "Notes here"
}
```

Update Contact

`PUT /contacts/:id`

Body: Same as create (partial update supported)

Delete Contact

`DELETE /contacts/:id`

Leads

List Leads

`GET /leads?search=&status=&source=&type=`

Query params:

- `search` — Search in leadNumber, company, notes
- `status` — new/attempting/connected/qualified/disqualified/converted
- `source` — google_ads/meta_ads/website/inbound_call/referral/manual
- `type` — mql/sql/deal

Get Lead

`GET /leads/:id`

Create Lead

`POST /leads`

Body:

```
{
  "contactId": 1,
  "createdById": 1,
  "source": "google_ads",
  "type": "mql",
  "company": "TechVista",
  "notes": "Notes",
  "expectedValue": 150000,
  "nextFollowUp": "2026-10-20T00:00:00.000Z"
}
```

Update Lead

`PUT /leads/:id`

Body: Partial update

Delete Lead

`DELETE /leads/:id`

Qualify Lead

`POST /leads/:id/qualify`

Body:

```
{
  "type": "sql",
  "qualificationNotes": "High budget, decision maker confirmed",
  "contactEmail": "new@example.com",
  "contactFirstName": "John",
  "contactLastName": "Doe"
}
```

If type is sql/deal and contact doesn't exist, creates a new contact.

Convert Lead

`POST /leads/:id/convert`

Body:

```
{
  "notes": "Deal notes"
}
```

Deals

List Deals

`GET /deals?search=&status=&stageId=&pipelineId=`

Get Deal

`GET /deals/:id`

Create Deal

`POST /deals`

Body:

```
{
  "dealName": "TechVista CRM",
  "amount": 150000,
  "contactId": 1,
  "stageId": 1,
  "pipelineId": 1,
  "createdById": 1,
  "probability": 80,
  "expectedCloseDate": "2026-11-01",
  "notes": "Notes"
}
```

Update Deal

`PUT /deals/:id`

Delete Deal

`DELETE /deals/:id`

Move Deal Stage

`PATCH /deals/:id/stage`

Body:

```
{ "stageId": 2 }
```

Automatically updates deal status based on stage (won/lost).

Pipelines

List Pipelines

`GET /pipelines`

Get Pipeline

`GET /pipelines/:id`

Create Pipeline

`POST /pipelines`

Body:

```
{
  "name": "New Pipeline",
  "description": "Description",
  "isDefault": false,
  "stages": [
    { "name": "Stage 1", "position": 0, "color": "#6366f1" }
  ]
}
```

Update Pipeline

`PUT /pipelines/:id`

Delete Pipeline

`DELETE /pipelines/:id` — Fails if deals exist

Add Stage

`POST /pipelines/:pipelineId/stages`

Body:

```
{ "name": "New Stage", "color": "#6366f1", "position": 0, "isClosedWon": false, "isClosedLost": fa
```

Update Stage

`PUT /pipelines/stages/:id`

Delete Stage

`DELETE /pipelines/stages/:id` — Fails if deals exist

Dashboard

`GET /dashboard`

Returns aggregated metrics:

```
{
  "totalContacts": 8,
  "totalLeads": 8,
  "mqlCount": 5,
  "sqlCount": 2,
  "openDeals": 5,
  "openPipelineValue": 555000,
  "closedWonRevenue": 300000,
  "closedLostValue": 60000,
  "leadsBySource": [...],
  "leadsByStatus": [...],
  "dealsByStage": [...],
  "recentLeads": [...],
  "recentActivities": [...],
  "leadsOverTime": [...],
  "conversionRate": { "total": 8, "converted": 1, "rate": 13 }
}
```

Activities

List

`GET /activities?contactId=&leadId=&dealId=&type=&completed=`

Get

`GET /activities/:id`

Create

`POST /activities`

Body:

```
{
  "type": "call",
  "subject": "Discovery call",
  "description": "Discussed requirements",
  "dueDate": "2026-10-20",
  "priority": "high",
  "contactId": 1,
  "leadId": null,
  "dealId": null,
  "createdById": 1
}
```

Update

`PUT /activities/:id`

Delete

`DELETE /activities/:id`

Simulated Sources

All return the created Lead with associated Contact.

- `POST /sources/google-ads` — Simulated Google Ads lead
- `POST /sources/meta-ads` — Simulated Meta Ads lead
- `POST /sources/website-form` — Simulated website form lead
- `POST /sources/inbound-call` — Simulated inbound call lead

Reports

`GET /reports`

Returns:


```
{
  "leadsBySource": [{"source": "google_ads", "_count": 2}],
  "leadsByStatus": [{"status": "new", "_count": 2}],
  "dealsByStage": [{"stageId": 1, "_count": 3, "_sum": {"amount": 150000}}],
  "wonLost": [{"status": "won", "_count": 1, "_sum": {"amount": 300000}}],
  "monthlyRevenue": [{"month": "2026-10", "revenue": 300000}]
}
```

PART III — FRONTEND

JavaScript Guide

ES6+ Features Used

Arrow Functions

```
// Component definition
export default function Dashboard() { ... }

// Callback
const handleClick = () => { ... }

// Array methods
items.map(item => <div key={item.id}>{item.name}</div>)
```

Destructuring

```
// Props destructuring
function Card({ title, value, icon: Icon, color = 'indigo' }) { ... }

// Array destructuring
const [firstName, lastName] = names[idx].split(' ')

// Object destructuring
const { totalContacts, openDeals } = dashboardData
```

Template Literals

```
const url = `${API_BASE}${endpoint}`
<p className={`text-sm ${active ? 'font-bold' : ''}`}>Label</p>
```

Spread Operator

```
// Merge objects
const where = { ...(search ? { OR: [...] } : {}) }

// Pass props
<button {...props} />
```

Array Methods

```
// map - transform arrays to JSX
contacts.map(c => <tr key={c.id}>...</tr>)

// filter - select subset
const stageDeals = deals.filter(d => d.stageId === stage.id)

// reduce - aggregate
const stageValue = stageDeals.reduce((sum, d) => sum + d.amount, 0)

// find - locate element
const selectedPipeline = pipelines.find(p => p.id === Number(pipelineFilter))

// forEach - iterate
activitySubjects.forEach((a, i) => { ... })
```

Async/Await

```
const loadContacts = async () => {
  setLoading(true)
  try {
    const data = await api.getContacts(params)
    setContacts(data)
  } catch (e) {
    console.error('Failed to load:', e)
  }
  setLoading(false)
}
```

Modules

```
// Named exports
export default function Dashboard() { ... }
export function CardSkeleton() { ... }

// Named imports
import { useState, useEffect } from 'react'
import { api } from '../services/api'
import { formatCurrency } from '../utils/constants'

// Default imports
import Button from '../ui/Button'
```

Optional Chaining & Nullish Coalescing

```
const value = data?._sum?.amount || 0
const name = lead?.contact?.firstName || 'Unknown'
```

Promises and Error Handling

```
// Promise chains (used in route handlers)
router.get('/', (req, res) => {
  getContacts(req.query)
    .then(data => res.json(data))
    .catch(err => res.status(500).json({ error: err.message }))
})

// Async/await with try/catch (used in components)
try {
  await api.createContact(form)
  toast.success('Created!')
} catch (err) {
  setError(err.message)
}
```

Event Handling

```
// Click handlers
<button onClick={() => setModalOpen(true)}>Open</button>
<button onClick={handleSubmit}>Submit</button>

// Form submission
<form onSubmit={handleSubmit}>
  <input onChange={e => setForm({ ...form, name: e.target.value })} />
</form>

// Event delegation (table row click with button stopPropagation)
<tr onClick={() => viewContact(c)}>
  <button onClick={e => { e.stopPropagation(); editContact(c) }}>Edit</button>
</tr>
```

Conditional Rendering

```
// Ternary
{loading ? <Skeleton /> : <DataTable />}

// Short-circuit
{error && <div className="text-red-600">{error}</div>}

// Map with conditional
{items.map(item => (
  <div key={item.id} className={item.active ? 'bg-green-50' : 'bg-gray-50'}>
    {item.active ? <ActiveBadge /> : <InactiveBadge />}
  </div>
))}
```

React Guide

Core Concepts

What is React?

React is a JavaScript library for building user interfaces. It lets you compose UI pieces called "components" and efficiently update the DOM when data changes.

JSX (JavaScript XML)

JSX looks like HTML but runs inside JavaScript:

```
// This JSX:
<div className="card">
  <h2>{title}</h2>
  <p>{description}</p>
</div>

// Gets compiled to:
React.createElement('div', { className: 'card' },
  React.createElement('h2', null, title),
  React.createElement('p', null, description)
)
```

Components

Components are reusable building blocks. Each component is a function that returns JSX:

```
function Button({ children, onClick, variant }) {
  return <button onClick={onClick} className={variant}>{children}</button>
}
```

Props (Properties)

Props pass data from parent to child:

```
// Parent passes props
<Card title="Total Leads" value={42} icon={Users} color="indigo" />

// Child receives props
function Card({ title, value, icon: Icon, color }) {
  return (
    <div>
      <p>{title}</p>
      <p>{value}</p>
      {Icon && <Icon className="w-5 h-5" />}
    </div>
  )
}
```

State (useState)

State is a component's internal memory. When state changes, React re-renders:

```
const [contacts, setContacts] = useState([])    // Array state
const [loading, setLoading] = useState(true)     // Boolean state
const [modalOpen, setModalOpen] = useState(false) // Toggle state
const [form, setForm] = useState({ name: '' })   // Object state
```

useEffect Hook

`useEffect` runs code after a render. Common patterns:

Data fetching:

```
useEffect(() => {
  async function load() {
    const data = await api.getDashboard()
    setData(data)
  }
  load()
}, []) // Empty array = run once on mount
```

Dependency array:

```
useEffect(() => {
  loadContacts()
}, [search, statusFilter]) // Re-run when these change
```

Controlled Forms

Form inputs whose values are controlled by React state:

```
<input
  value={form.firstName}
  onChange={e => setForm({ ...form, firstName: e.target.value })}
/>
```

Conditional Rendering

Show different UI based on conditions:

```
{loading && <Skeleton />}
{error && <Error message={error} />}
{data.length === 0 ? <Empty /> : <List items={data} />}
```

Lists and Keys

Render arrays with `.map()` and provide unique keys:

```
{contacts.map(contact => (
  <tr key={contact.id}>
    <td>{contact.firstName}</td>
  </tr>
))}
```

React Router (react-router-dom v7)

Route Configuration

```
<Routes>
  <Route path="/" element={<AppShell><Dashboard /></AppShell>} />
  <Route path="/contacts" element={<AppShell><Contacts /></AppShell>} />
  <Route path="*" element={<AppShell><NotFound /></AppShell>} />
</Routes>
```

Navigation Links

```
import { NavLink } from 'react-router-dom'

<NavLink to="/contacts" className={({ isActive }) =>
  isActive ? 'bg-primary-50 text-primary-700' : 'text-surface-600'
}>
  Contacts
</NavLink>
```

Breadcrumbs

```
import { useLocation } from 'react-router-dom'
const location = useLocation()
// location.pathname gives current route
```

Component Reuse

UI Components (src/components/ui/)

Component	Purpose
Button	Consistent button styling with variants
Badge	Status indicators with color coding
Card	KPI display card with icon
Modal	Reusable modal dialog
Input	Input, Select, Textarea with labels and error states
SearchableSelect	Dropdown with search/filter

Pattern: Props + Composition

```
// Generic Modal accepts any children
<Modal isOpen={open} onClose={close} title="Edit Contact">
  <form onSubmit={handleSubmit}>
    <Input label="Name" ... />
    <Button type="submit">Save</Button>
  </form>
</Modal>

// Same Modal for confirmations
<Modal isOpen={showConfirm} onClose={close} title="Confirm">
  <p>Are you sure?</p>
  <Button onClick={onConfirm}>Yes</Button>
</Modal>
```

State Management in This App

No Redux/Zustand — uses local state only:

- Each page component manages its own data state
- `useState` + `useEffect` for data fetching
- Props for passing callbacks between components
- Simple and sufficient for this application scale

Key React Concepts Demonstrated

Concept	Where Used
useState	Every page component
useEffect	Data loading in Dashboard, Contacts, Leads, Deals, Activities, Reports
Props	All component composition
JSX	Entire application
Component composition	AppShell wrapping page content
Conditional rendering	Loading states, empty states, modals
List rendering	Table rows, cards, dropdown options
Event handlers	onClick, onChange, onSubmit
Controlled components	All form inputs
useRef	SearchableSelect dropdown
React Router	App.jsx route definitions

Frontend Guide

File Structure

```
src/
  components/
    layout/
      AppShell.jsx      — Main layout wrapper (sidebar + header + content)
      Sidebar.jsx       — Navigation sidebar with collapse
      Header.jsx        — Top bar with breadcrumbs, search, profile
    ui/
      Button.jsx        — Reusable button (primary/secondary/danger/ghost/success)
      Badge.jsx         — Status badges with colors
      Card.jsx          — KPI card for dashboard + CardSkeleton loader
      Modal.jsx         — Modal dialog + ConfirmDialog
      Input.jsx         — Input, Select, Textarea (named exports + default)
      SearchableSelect.jsx — Searchable dropdown
    common/
      NotFound.jsx      — 404 page
    dashboard/Dashboard.jsx
    contacts/Contacts.jsx
    leads/Leads.jsx
    deals/Deals.jsx
    pipelines/Pipelines.jsx
    sources/Sources.jsx
    activities/Activities.jsx
    reports/Reports.jsx
  services/api.js       — Centralized API client
  utils/constants.js    — Constants, formatters, helpers
  App.jsx               — Route definitions
  main.jsx              — Entry point (BrowserRouter + Toaster)
  index.css             — Tailwind imports + custom theme + scrollbar
```

Component Patterns

Data Loading Pattern

```
// Every page component follows this pattern:
const [data, setData] = useState([])
const [loading, setLoading] = useState(true)

const load = async () => {
  setLoading(true)
  try {
    const result = await api.getXxx()
    setData(result)
  } catch (e) {
    console.error(e)
  }
  setLoading(false)
}

useEffect(() => { load() }, [dependencies])
```

Form Pattern

```
const [form, setForm] = useState(initialValues)
const handleSubmit = async (e) => {
  e.preventDefault()
  try {
    await api.createXxx(form)
    setModalOpen(false)
    loadData()
  } catch (err) {
    setError(err.message)
  }
}
```

Search/Filter Pattern

```
const [search, setSearch] = useState('')
useEffect(() => { loadData() }, [search, otherFilters])
```

Styling Conventions

- Use Tailwind utility classes
- Color tokens: `surface-50` to `surface-900`, `primary-50` to `primary-900`
- Spacing: standard Tailwind scale
- Border radius: `rounded-lg` (8px) for cards, `rounded-xl` (12px) for containers
- Shadows: `shadow-sm` for cards, `shadow-md` on hover, `shadow-lg` for modals
- Transitions: `transition-colors` or `transition-shadow`

Design Tokens (index.css)

```
--color-primary-500: #6366f1    /* Indigo accent */
--color-primary-600: #4f46e5    /* Primary button/hover */
--color-surface-50:  #f8fafc    /* Page background */
--color-surface-100: #f1f5f9    /* Alternating rows */
--color-surface-200: #e2e8f0    /* Borders */
--color-surface-500: #64748b    /* Secondary text */
--color-surface-700: #334155    /* Headers */
--color-surface-800: #1e293b    /* Primary text */
```

Recharts Usage

```
import { ResponsiveContainer, LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip } from 'recha

<ResponsiveContainer width="100%" height={260}>
  <LineChart data={chartData}>
    <CartesianGrid strokeDasharray="3 3" stroke="#e2e8f0" />
    <XAxis dataKey="date" tick={{ fontSize: 11, fill: '#94a3b8' }} />
    <YAxis tick={{ fontSize: 11, fill: '#94a3b8' }} />
    <Tooltip contentStyle={{ borderRadius: '8px', border: '1px solid #e2e8f0', fontSize: '13px' }}
    <Line type="monotone" dataKey="count" stroke="#6366f1" strokeWidth={2} />
  </LineChart>
</ResponsiveContainer>
```

Responsive Breakpoints

- `sm:` — 640px (tablet)
- `md:` — 768px (large tablet)
- `lg:` — 1024px (desktop)

Grid examples:

```
// Cards: 1 col mobile → 2 col tablet → 4 col desktop
className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4"

// Charts: 1 col mobile → 2 col desktop
className="grid grid-cols-1 lg:grid-cols-2 gap-6"
```

Icons (Lucide React)

All icons imported from `lucide-react`:

```
import { Users, Plus, Search, Edit, Trash2, Phone, Mail } from 'lucide-react'
// Usage: <Users className="w-5 h-5" />
```

Standard icon sizes:

- Navigation: `w-5 h-5`
- Buttons: `w-4 h-4`
- Cards: `w-5 h-5`

API Client Pattern

All API calls go through `src/services/api.js`:

```
import { api } from '../services/api'

// Simple get
const data = await api.getDashboard()

// With params
const contacts = await api.getContacts({ search, sortBy })

// POST/PUT with body
const newLead = await api.createLead(formData)

// Delete
await api.deleteContact(id)
```

The client handles:

- Base URL prefix (`/api`)
- JSON serialization
- Error propagation
- Consistent interface

PART IV — OPERATIONS

Setup and Run

Prerequisites

- Node.js 18+ (tested with v24.14.0)
- npm 9+ (tested with v11.21.0)

Step-by-Step Setup

1. Clone/Open the Project

```
cd C:/Users/Nischay\ r\ gowda/MY_PROJECTS/crm-platform
```

2. Install Dependencies

```
npm install
```

3. Configure Environment

```
echo DATABASE_URL="file:./server/prisma/dev.db" > .env
```

4. Push Database Schema

```
npx prisma db push --schema server/prisma/schema.prisma
```

5. Seed Sample Data

```
node server/prisma/seed.js
```

Expected output: `Database seeded successfully!`

6. Start the Application

Option A: Run both servers together

```
npm run dev:all
```

Option B: Run separately (two terminals)

Terminal 1 — Backend:

```
npm run server
```

Terminal 2 — Frontend:

```
npm run dev
```

7. Open in Browser

Navigate to `http://localhost:5173`

Troubleshooting

Port Already in Use

```
netstat -ano | findstr :5000
taskkill //PID <PID> //F

netstat -ano | findstr :5173
taskkill //PID <PID> //F
```

Prisma Client Not Generated

```
npx prisma generate --schema server/prisma/schema.prisma
```

Database Issues (Reset)

```
del server\prisma\dev.db

npx prisma db push --schema server/prisma/schema.prisma

node server/prisma/seed.js
```

Build Errors

```
rm -rf node_modules/.vite
npx vite build
```

Production Build

```
npm run build
npm run preview
```

The built files will be in `dist/`.

Database Studio (Optional)

```
npx prisma studio --schema server/prisma/schema.prisma
```

Available Scripts

Script	Command	Description
dev	`npm run dev`	Vite dev server (port 5173)
server	`npm run server`	Express backend (port 5000)
dev:all	`npm run dev:all`	Both servers concurrently
build	`npm run build`	Production build
preview	`npm run preview`	Preview production build
lint	`npm run lint`	ESLint check
db:push	`npm run db:push`	Push schema to SQLite
db:seed	`npm run db:seed`	Seed sample data
db:studio	`npm run db:studio`	Open Prisma Studio

Testing Report

Environment

- OS: Windows 11 Home Single Language
- Node.js: v24.14.0
- npm: v11.21.0
- React: 19.2.8
- Vite: 8.3.0
- Prisma: 6.8.0
- SQLite: bundled with Prisma

Test Results

Build & Lint

Test	Status	Notes
`npm run build`	PASS	Production build succeeds, ~1.6s, 762 kB JS / 38 kB CSS
`npm run lint`	PASS	0 errors, 0 warnings across all source files
`npm run check:routes`	PASS	All 38 frontend API calls resolve to registered Express routes

Backend Tests

Test	Status	Notes
Server starts	PASS	Express server on port 5000
Health endpoint	PASS	`GET /api/health` returns `{"status":"ok"}`
Dashboard endpoint	PASS	Returns 8 KPIs, charts data, recent items
Contacts CRUD	PASS	Create, read, update, delete all work
Leads CRUD	PASS	Create, read, update, delete all work
Deals CRUD	PASS	Create, read, update, delete all work
Lead qualification	PASS	MQL→SQL qualification with contact creation
Simulate Google Ads	PASS	Creates contact + MQL lead with source=google_ads
Simulate Meta Ads	PASS	Creates contact + MQL lead with source=meta_ads
Simulate Website	PASS	Creates contact + MQL lead with source=website
Simulate Inbound Call	PASS	Creates contact + MQL lead with source=inbound_call
Pipeline CRUD	PASS	Create pipeline with stages
Stage CRUD	PASS	Create, update, delete stages
Deal stage movement	PASS	Moves deal between stages, updates status
Dashboard calculations	PASS	Correctly distinguishes open/won/lost
Reports endpoint	PASS	Returns source, status, stage, revenue data
Duplicate email prevention	PASS	Returns error on duplicate contact email
Stage deletion safety	PASS	Returns error when stage has deals

Frontend Tests

Test	Status	Notes
Vite dev server	PASS	Serves on port 5173
React Router navigation	PASS	All 8 routes resolve correctly
Sidebar navigation	PASS	Active page highlighting
404 page	PASS	Shows with navigation links
Dashboard renders	PASS	8 KPI cards + 4 charts + recent sections
Contacts page renders	PASS	Table with seeded data
Leads page renders	PASS	Table with 8 seeded leads
Deals page renders	PASS	Kanban board with 7 seeded deals
Pipelines page renders	PASS	Expandable pipeline cards
Sources page renders	PASS	4 source cards with simulate buttons
Activities page renders	PASS	Activity list with 7 seeded activities
Reports page renders	PASS	4 charts with data
Hot reload	PASS	Vite HMR functional
Toast notifications	PASS	Success/error toasts display

Database Tests

Test	Status	Notes
Schema created	PASS	All 7 tables created
Seed data	PASS	8 contacts, 8 leads, 7 deals, 7 stages, 7 activities
Relationships	PASS	Foreign keys working correctly
Data persistence	PASS	Survives server restart

End-to-End Demo Verification

Step	Expected	Actual
1. Open dashboard	See 8 KPIs, charts, recent items	PASS
2. Simulate Google Ads lead	New MQL lead + contact created	PASS
3. Find lead in Leads page	Visible in table with source=Google Ads	PASS
4. Open its contact	Contact details show	PASS
5. Explain source and MQL status	Correct badges displayed	PASS
6. Qualify lead to SQL	Status changes, contact created/linked	PASS
7. Create deal	Deal in pipeline	PASS
8. Move deal through stages	Stage changes, status updates	PASS
9. Mark Closed Won	Deal status=won, actualCloseDate set	PASS
10. Dashboard updates	Figures reflect new data	PASS

Known Issues

- No authentication (documented limitation)
- Simulated integrations are demonstrations only
- CSV export exports raw data, not formatted charts
- Large datasets may need pagination (not implemented)

Dashboard Accuracy (arithmetic cross-check)

Every KPI was recomputed from the raw record sets and compared to the API response. All values matched exactly.

Metric	Verification method	Result
`totalContacts`	Compared to `contacts.length`	PASS (22 = 22)
`totalLeads`	Compared to `leads.length`	PASS
`mqlCount`	Compared to `leads.filter(type === 'mql').length`	PASS
`sqlCount`	Compared to `leads.filter(type === 'sql').length`	PASS
`openDeals`	Compared to `deals.filter(status === 'open').length`	PASS
`openPipelineValue`	Compared to sum of open-deal amounts	PASS (805000)
`closedWonRevenue`	Compared to sum of won-deal amounts	PASS
`closedLostValue`	Compared to sum of lost-deal amounts	PASS (585000)
Won excluded from open	Confirmed open value does not include won deals	PASS
Lost excluded from won	Confirmed won revenue unchanged when a deal is lost	PASS

A deal was created, marked won, and the dashboard re-read: closed-won revenue rose by exactly the deal amount while open pipeline value stayed flat, then dropped back when the deal was marked lost. No revenue is ever double-counted.

Route Parity

`npm run check:routes` parses `server/server.js` mounts plus every `router.<verb>()` declaration and compares them against every `apiRequest()` call in `src/services/api.js`. Result: 38 routes, 38 calls, 38 matches. This catches the failure mode where a UI button silently 404s because an endpoint was renamed.

Persistence

Server was stopped and restarted with no other changes. All counts and totals were byte-identical before and after: contacts=26, leads=19, open=805000, won=800000, lost=585000, deals=12.

Untested

- Mobile responsive testing (not verified on real physical devices)
- Browser automation / component-level tests (no test runner installed; adding

one was out of scope for the deadline)

- Production build served to a real browser for visual inspection

Note on an earlier entry: this report previously listed lint as NOT TESTED. Lint has since been run and passes clean; the row above reflects that.

Known Limitations

Authentication

- **No authentication or authorization** — All APIs are open. This is documented as a known limitation. Adding auth would require session management (JWT or sessions), protected routes, and user-specific data isolation.

Lead Source Simulations

- All 4 lead source integrations are **simulated demonstrations**. They do not connect to Google Ads, Meta Ads, website forms, or telephony systems. Real integrations would require OAuth, webhooks, API keys, and third-party SDKs.

Email/Notifications

- No real email sending. Activities of type "email" are logged records only.
- Notifications table exists in the database but has no frontend display component.

Data Scale

- No pagination — all records loaded at once. Acceptable for semester demo but not production scale.
- No lazy loading or infinite scroll.
- SQLite is suitable for development but not recommended for production multi-user deployment.

Deal Movement

- Kanban board uses button-based stage movement (Back/Next). Drag-and-drop was considered but excluded for reliability within the deadline.

Reporting

- Basic charts only. No date range filtering, no comparison views, no scheduled reports.
- CSV export is minimal — raw data only.

Search

- Simple substring matching. No full-text search, no fuzzy matching, no search suggestions.

Import/Export

- No data import (CSV, Excel). Only CSV export for reports.
- No backup/restore functionality.

Multi-tenancy

- Single-tenant only. No organization/workspace isolation.

Mobile

- Layout is responsive but not fully optimized for mobile use. CRM workflows are primarily desktop-oriented.

Audit Logging

- No audit trail of who changed what and when (beyond Prisma's updatedAt timestamps).

Integrations

- No calendar integration, no email sync, no document storage, no third-party CRM sync.

Future Roadmap

Phase 1: Authentication & Security

- [ ] JWT-based authentication
- [ ] Role-based access control (Admin, Manager, Sales Rep, Viewer)
- [ ] Password reset flow
- [ ] Protected API routes
- [ ] User-specific data isolation

Phase 2: Enhanced Lead Management

- [ ] Lead scoring algorithm
- [ ] Lead routing/assignment rules
- [ ] Duplicate lead detection
- [ ] Bulk import leads (CSV)
- [ ] Lead enrichment (auto-fill company info)
- [ ] Email tracking and sequences

Phase 3: Advanced Pipeline

- [ ] Drag-and-drop Kanban (react-dnd or @dnd-kit)
- [ ] Multiple pipeline templates
- [ ] Forecast vs actual reporting
- [ ] Deal probability auto-calculation
- [ ] Commission tracking

Phase 4: Communication

- [ ] Real email sending (Nodemailer + SMTP)
- [ ] Email templates
- [ ] Click tracking and open rates
- [ ] SMS integration
- [ ] In-app notifications with read/unread

Phase 5: Reporting & Analytics

- [ ] Custom date range filters
- [ ] Custom report builder

- [ ] Sales forecasting
- [ ] Team performance metrics
- [ ] Export to PDF
- [ ] Dashboard customization

Phase 6: Integrations

- [ ] Google Ads API integration
- [ ] Meta Ads API integration
- [ ] Website form webhook handler
- [ ] Calendar sync (Google Calendar)
- [ ] Zapier/Make.com webhooks
- [ ] REST API for external integrations

Phase 7: Advanced Features

- [ ] Document management
- [ ] Contract generation
- [ ] Invoice creation
- [ ] Payment tracking
- [ ] Customer portal
- [ ] Mobile app (React Native)
- [ ] Dark mode
- [ ] Multi-language support

Phase 8: Infrastructure

- [ ] PostgreSQL migration
- [ ] Docker containerization
- [ ] CI/CD pipeline
- [ ] Cloud deployment (Vercel + Railway/Render)
- [ ] Monitoring and logging
- [ ] Automated backups

PART V — PRESENTATION

Final Presentation Outline

Presentation Structure (~15-20 minutes)

Slide 1: Title

- Project: Nexus CRM
- Student name and ID
- Course: Full Stack Development 1
- University and semester

Slide 2: Problem Statement

- Why CRMs are important for businesses
- Gap: Small businesses can't afford enterprise CRMs
- Solution: A simple, effective, open-source CRM

Slide 3: Product Workflow

- Visual flow: Lead Sources → MQL → SQL → Deal → Pipeline → Close
- Screenshot of Dashboard

Slide 4: Technology Stack

- Frontend: React, Vite, Tailwind CSS, Recharts, Lucide React
- Backend: Express.js
- Database: SQLite + Prisma
- Why each technology was chosen

Slide 5: Architecture

- System architecture diagram
- Frontend/backend separation
- Data flow diagram

Slide 6: Database Design

- ERD diagram
- Key relationships explained
- Why relational database was chosen

Slide 7: Dashboard Module

- Live demo: Open dashboard

- Explain 8 KPIs and how they're calculated
- Explain charts (Recharts)
- How data flows from database → API → frontend

Slide 8: Lead Management

- MQL vs SQL vs Deal
- Lead lifecycle
- Live demo: Simulate a lead, view in table

Slide 9: Contact Management

- CRUD operations
- Duplicate prevention
- Lifecycle stages
- Live demo

Slide 10: Deals & Pipeline

- Kanban board demo
- Stage movement
- Deal tracking
- Live demo: Create deal, move through stages

Slide 11: Lead Sources (Simulated)

- Explain each simulation
- How contacts are auto-created/linked
- Live demo

Slide 12: Code Walkthrough

- Show 2-3 key files
- Explain data flow through one feature end-to-end
- Example: Dashboard → API → Controller → Prisma → Database

Slide 13: Challenges & Solutions

- Prisma schema design decisions
- Frontend state management approach
- Build/import troubleshooting
- Design system implementation

Slide 14: Testing & Results

- Test coverage summary
- End-to-end demo results
- Known issues

Slide 15: Future Enhancements

- Roadmap highlights
- Authentication
- Real integrations
- Mobile app

Slide 16: Thank You / Q&A

- Thank you slide
- GitHub link
- Live demo URL

Demo Script (5-7 minutes)

Part 1: Dashboard (1 min)

1. Open `http://localhost:5173`
2. Point out KPI cards — explain each metric
3. Show charts — lead acquisition, source distribution

Part 2: Simulate Lead (1 min)

1. Navigate to Lead Sources
2. Click "Simulate Google Ads Lead"
3. Show success toast
4. Point out what was created (contact + lead)

Part 3: View & Qualify Lead (2 min)

1. Navigate to Leads
2. Find the new lead
3. Explain MQL status and source
4. Click Qualify → select SQL → fill contact info → submit
5. Show status updated to SQL

Part 4: Create Deal & Pipeline (2 min)

1. Navigate to Deals
2. Create a new deal for the lead

3. Show deal in Kanban board
4. Move deal through stages (Next button)
5. Show stage totals update

Part 5: Dashboard Update (1 min)

1. Return to Dashboard
2. Show updated metrics reflecting new lead and deal
3. Charts reflect new data

Key Talking Points

Technical

- Explain why React hooks (useState/useEffect) are used
- Walk through one API call: component → api.js → Express route → controller → Prisma → SQLite
- Explain Tailwind CSS design system approach
- Explain Recharts for data visualization

CRM Concepts

- MQL = Marketing Qualified Lead (generated from marketing efforts)
- SQL = Sales Qualified Lead (verified by sales team)
- Deal = Active sales opportunity with expected value
- Pipeline = Visual representation of deals in various stages
- Conversion rate = Converted leads / Total leads

Design Decisions

- Why Tailwind CSS over CSS modules (consistency, speed, design system)
- Why SQLite (zero-config, perfect for semester project)
- Why no auth (documented limitation, kept scope manageable)
- Why simulated integrations (real APIs require accounts, keys, rate limits)

Viva Question Bank

100+ Questions with Answers

General / Introduction

Q1. What is Nexus CRM? A: Nexus CRM is a full-stack Customer Relationship Management web application. It helps businesses manage contacts, leads, deals, and sales pipelines. It's built with React (frontend), Express.js (backend), and SQLite with Prisma ORM (database).

Q2. What problem does this CRM solve? A: It helps small and medium businesses track leads, manage contacts, and visualize their sales pipeline. Instead of using spreadsheets or multiple disconnected tools, everything is in one place.

Q3. What technologies did you use and why? A:

- React — For building a dynamic, component-based UI with efficient re-rendering
- Vite — Fast build tool with hot module replacement during development
- Tailwind CSS — Utility-first CSS framework for rapid, consistent styling
- Recharts — React charting library for dashboard visualizations
- Lucide React — Clean, consistent icon library
- Express.js — Minimal, flexible Node.js backend framework
- Prisma ORM — Type-safe database access with auto-generated queries
- SQLite — Zero-configuration file-based database, perfect for development

HTML/CSS/Tailwind

Q4. What is JSX? A: JSX is a syntax extension for JavaScript that lets you write HTML-like code inside JavaScript files. For example: `<div className="card"><h2>{title}</h2></div>`. It gets compiled to `React.createElement()` calls.

Q5. Why use className instead of class in JSX? A: Because `class` is a reserved keyword in JavaScript. JSX uses `className` which maps to the HTML `class` attribute.

Q6. What is Tailwind CSS? A: Tailwind CSS is a utility-first CSS framework. Instead of writing custom CSS classes, you compose designs using pre-built utility classes like `flex`, `p-4`, `text-lg`, `bg-blue-500`. This ensures consistency and speeds up development.

Q7. How does the CSS work in this project? A: We use Tailwind CSS v4 with the Vite plugin. Custom design tokens (colors like `primary-500`, `surface-800`) are defined in `index.css` using CSS custom properties. Tailwind generates utility classes from these tokens.

Q8. What is the difference between px, py, gap, and space? A:

- `px-4` = padding-left + padding-right = 16px
- `py-4` = padding-top + padding-bottom = 16px

- `gap-4` = space between flex/grid items = 16px
- `space-y-4` = vertical space between direct children

Q9. How does the responsive design work? A: Using Tailwind's responsive prefixes:

- `sm:` applies at 640px+
- `md:` applies at 768px+
- `lg:` applies at 1024px+

Example: `grid-cols-1 sm:grid-cols-2 lg:grid-cols-4` means 1 column on mobile, 2 on tablet, 4 on desktop.

JavaScript

Q10. What are arrow functions? A: Arrow functions are a shorter syntax for writing functions in ES6: `const add = (a, b) => a + b`. They also inherit `this` from the enclosing scope. Used throughout the project for callbacks and component definitions.

Q11. What is destructuring? A: Destructuring extracts values from arrays or objects into variables. Example: `const [contacts, setContacts] = useState([])` extracts the state value and setter. `const { name, email } = contact` extracts properties from an object.

Q12. What is the spread operator? A: The spread operator (`...`) expands an array or object. Used to merge objects: `{ ...form, firstName: 'New' }` creates a new object with all of form's properties plus a changed firstName.

Q13. What are template literals? A: Template literals use backticks (`` ` ``) and allow embedded expressions with `${}`. Example: `` `Hello, ${name}!` ``. Also supports multi-line strings.

Q14. What are promises and async/await? A: Promises represent async operations. `async/await` is syntactic sugar for working with promises. Example:

```
async function load() {
  const data = await api.getDashboard() // waits for response
  setData(data)
}
```

Q15. What is the fetch API? A: The `fetch()` API makes HTTP requests. It returns a Promise that resolves to a Response object. We use it in `src/services/api.js` to communicate with the backend:

```
const response = await fetch('/api/contacts')
const data = await response.json()
```

Q16. How does error handling work in this app? A: We use try/catch blocks around all async operations. API errors are caught and displayed to users via error messages or toast notifications. Backend errors return `{ error: "message" }` with appropriate HTTP status codes.

React

Q17. What is React? A: React is a JavaScript library for building user interfaces using components. It uses a virtual DOM for efficient updates and follows a declarative programming model.

Q18. What are React components? A: Components are reusable pieces of UI. In this project, we use functional components — JavaScript functions that return JSX. Examples: `Dashboard`, `Contacts`, `Button`, `Card`.

Q19. What are props? A: Props (properties) are how data flows from parent to child components. Example: `<Card title="Total Leads" value={42} icon={Users} />` passes title, value, and icon to the Card component.

Q20. What is JSX? (See Q4) A: JSX is a syntax extension that allows writing HTML-like code in JavaScript. It's compiled to React.createElement() calls.

Q21. What is state in React? A: State is a component's internal memory. When state changes, React re-renders the component. We use `useState` hook: `const [contacts, setContacts] = useState([])`.

Q22. What is the useState hook? A: `useState` is a React hook that adds state to functional components. It returns an array: `[currentValue, setterFunction]`. Calling the setter triggers a re-render.

Q23. What is the useEffect hook? A: `useEffect` lets you perform side effects in components (data fetching, subscriptions, DOM manipulation). It runs after every render by default. With a dependency array, it runs only when those dependencies change.

Q24. What is the dependency array in useEffect? A: The second argument to useEffect. `useEffect(() => {...}, [dep1, dep2])` means the effect runs only when dep1 or dep2 changes. Empty array `[]` means run once (on mount).

Q25. What is conditional rendering? A: Showing different UI based on conditions. Examples:

- Ternary: `{loading ? <Spinner /> : <Content />}`
- Short-circuit: `{error && <Error />}`
- If/else inside JSX

Q26. What is the key prop in lists? A: The `key` prop helps React identify which items changed, were added, or removed. It should be a unique, stable identifier (like an ID). Example: `{items.map(item => <li key={item.id}>...</li>)}`.

Q27. What are controlled components? A: Form inputs whose values are controlled by React state. The value comes from state, and changes are handled by an onChange handler. Example: `<input value={form.name} onChange={e => setForm({...form, name: e.target.value})} />`.

Q28. What is React Router? A: React Router is a library for handling navigation in React apps. It maps URL paths to components. In this app, it maps `/`, `/contacts`, `/leads`, etc. to their respective page components.

Q29. What is the difference between BrowserRouter and HashRouter? A: BrowserRouter uses the HTML5 History API (clean URLs like `/contacts`). HashRouter uses URL hashes (`/#/contacts`). We use

BrowserRouter because the server is configured to serve the app for all routes.

Project-Specific

Q30. What is the application structure? A:

- `src/components/` — Reusable UI and page components
- `src/services/api.js` — Centralized API client
- `src/utils/constants.js` — Constants, formatters, helper functions
- `server/` — Express.js backend with routes and controllers
- `server/prisma/` — Prisma schema and seed data
- `docs/` — Documentation

Q31. How does data flow from the database to the Dashboard? A: 1. Dashboard component mounts, calls `api.getDashboard()`

1. api.js makes `fetch('/api/dashboard')`
2. Express route handler receives the request
3. Dashboard controller runs multiple Prisma queries (counts, aggregates, groupBys)
4. Controller returns compiled metrics
5. Route sends JSON response
6. Dashboard component receives data, updates state
7. React re-renders with new KPI values and chart data

Q32. How many API endpoints are there? A: Approximately 25+ endpoints across 8 route groups (contacts, leads, deals, pipelines, dashboard, activities, sources, reports) plus health check.

Q33. What is Prisma and why use it? A: Prisma is an ORM (Object-Relational Mapping) tool. It converts TypeScript/JavaScript code into database queries. Benefits: type-safe queries, auto-completion, migrations, and a clean data model definition language.

Q34. How does the lead qualification workflow work? A: A lead starts as MQL (Marketing Qualified Lead). When the sales team qualifies it, they use the qualify endpoint:

1. Set type to 'sql' (Sales Qualified Lead)
2. Optionally provide contact details (creates contact if not exists)
3. The lead's status updates to 'qualified'
4. When ready to become a deal, type changes to 'deal' and status to 'converted'

Q35. How are duplicate contacts prevented? A: The Contact model has a unique constraint on the email field. When creating or updating, the controller checks for existing emails and returns an error if a duplicate is found.

Q36. How does the Kanban pipeline work? A: The Deals page groups deals by pipeline stage. Each stage is a column showing stage name, deal count, and total value. Users move deals by clicking "Next" or "Back" buttons, which calls the stage movement API.

Q37. What are lead sources and why are they simulated? A: Lead sources represent where leads come from (Google Ads, Meta Ads, Website, Calls). They're simulated because real integrations require API keys, accounts, webhooks, and complex authentication — beyond a semester project scope. The simulation creates realistic-looking data.

Q38. How does the sidebar know which page is active? A: Using React Router's `useLocation()` hook. Each NavLink checks if the current pathname matches its path and applies an `active` class (primary background color).

Q39. What is the design system? A: A consistent set of colors (primary indigo, neutral surfaces), spacing scale, border radii, shadows, and typography defined in Tailwind config. All components use these tokens for visual consistency.

Q40. How does the search work in Contacts/Leads? A: The search input's value is stored in React state. It's passed as a query parameter to the API (`?search=...`). The backend uses Prisma's `contains` filter to match against relevant fields (name, email, company, lead number).

Backend

Q41. What is Express.js? A: Express.js is a minimalist web framework for Node.js. It provides routing, middleware, and request/response handling. In this project, it serves REST API endpoints that the frontend consumes.

Q42. What is REST API? A: REST (Representational State Transfer) is an architectural style for APIs. It uses HTTP methods (GET, POST, PUT, DELETE) on resource URLs. Our API follows REST: `GET /api/contacts`, `POST /api/contacts`, etc.

Q43. What is middleware in Express? A: Middleware functions that run before route handlers. In our server:

- `cors()` — Allows cross-origin requests from frontend
- `express.json()` — Parses JSON request bodies

Q44. How does the server handle errors? A: Each route handler wraps controller calls in try/catch. Errors are caught and sent as JSON with appropriate HTTP status codes (400 for bad requests, 404 for not found, 500 for server errors).

Q45. What is Prisma Client? A: The auto-generated query builder from the Prisma schema. It provides type-safe methods like `prisma.contact.findMany()`, `prisma.lead.create()`, etc.

Q46. What are Prisma relations? A: Defined in the schema with `@relation` attributes. Examples:

- Contact has many Leads (`contactId` foreign key)
- Deal belongs to Contact, Stage, Pipeline (three foreign keys)
- Activity optionally belongs to Contact, Lead, or Deal

Q47. What is Prisma seeding? A: The `seed.js` file populates the database with sample data. It creates users, contacts, leads, deals, pipeline stages, and activities. Run with `node server/prisma/seed.js`.

Q48. How does the database persist data? A: SQLite stores all data in a single file (`server/prisma/dev.db`). This file persists between server restarts. All CRUD operations through Prisma are written to this file.

Q49. What HTTP status codes does the API use? A:

- 200: Successful GET/PUT/DELETE
- 201: Successful POST (resource created)
- 400: Bad request (validation errors)
- 404: Resource not found
- 500: Server error

Q50. What is CORS and why is it needed? A: CORS (Cross-Origin Resource Sharing) allows the frontend (port 5173) to make requests to the backend (port 5000). Browsers block cross-origin requests by default. The `cors()` middleware adds appropriate headers.

Database

Q51. What is SQLite? A: SQLite is a file-based relational database. It doesn't require a server — the database is a single file. Perfect for development and small applications.

Q52. What is an ERD (Entity Relationship Diagram)? A: A diagram showing database tables and their relationships. In Nexus CRM: User→Lead, Contact→Lead/Deal/Activity, Pipeline→Stage→Deal, etc.

Q53. What are foreign keys? A: A foreign key is a column that references a primary key in another table. Example: `deal.contactId` references `contact.id`. This creates relationships between tables.

Q54. What is a unique constraint? A: A database constraint ensuring a field's value is unique across all rows. Example: `email` on Contact has `@unique`, preventing duplicate emails.

Q55. What is ON DELETE CASCADE? A: A foreign key behavior where deleting a parent record also deletes child records. Example: Deleting a Pipeline cascades to delete its Stages.

Q56. What is ON DELETE SET NULL? A: A foreign key behavior where deleting a parent record sets the child's foreign key to NULL. Example: Deleting a Contact sets `lead.contactId` to NULL (lead still exists but loses its contact).

Q57. What is Prisma's db push? A: A development command that creates/updates the database schema to match the Prisma schema definition. Unlike migrations, it doesn't create migration history files.

CRM Concepts

Q58. What is MQL? A: MQL stands for Marketing Qualified Lead. A lead that has shown interest through marketing efforts (ads, website visits, downloads) but hasn't been verified by sales.

Q59. What is SQL? A: SQL stands for Sales Qualified Lead. A lead that the sales team has verified as having genuine interest, budget, and authority to purchase.

Q60. What is a sales pipeline? A: A visual representation of where deals are in the sales process. Stages typically progress from "New Opportunity" through "Negotiation" to "Closed Won/Lost".

Q61. What is lead scoring? A: A methodology to rank leads based on their likelihood to convert. Not fully implemented in this MVP, but the qualification system is the foundation.

Q62. What is conversion rate? A: The percentage of leads that become customers. In Nexus CRM: `(converted leads / total leads) * 100`.

Q63. What is pipeline value? A: The total monetary value of all open (not yet won or lost) deals in the pipeline. Used for sales forecasting.

Q64. What is the difference between a contact and a lead? A: A contact is a person/company record (email, phone, etc.). A lead is a potential customer that goes through a qualification process. One contact can have multiple leads over time.

Code Quality

Q65. What coding conventions does the project follow? A:

- camelCase for variables and functions
- PascalCase for components
- Descriptive variable names
- Consistent file structure
- Component reusability
- Separation of concerns (routes, controllers, UI)

Q66. Why use functional components instead of class components? A: Functional components are simpler, support hooks, and are the modern React standard. They require less boilerplate code and are easier to test.

Q67. What is component composition? A: Building complex UIs by combining smaller components. Example: `AppShell` wraps every page with `Sidebar` + `Header` + content area.

Q68. Why keep API logic separate from UI? A: Separation of concerns makes the codebase maintainable. If the API endpoint changes, only `src/services/api.js` needs updating — not every component that uses it.

Challenges

Q69. What challenges did you face? A:

1. Prisma v8 RC configuration complexity — resolved by using stable Prisma v6
2. Import path consistency across component directories
3. Designing a scalable but simple state management approach
4. Making the UI look professional without a design system library

Q70. How did you handle the Prisma version issues? A: Initially installed Prisma v8 RC which requires a new config format. Downgraded to Prisma v6.8.0 which uses the familiar `schema.prisma` with `url` in

datasource.

Q71. What was the hardest feature to implement? A: The qualification workflow — it needed to handle contact creation/linking, status changes, and type transitions while maintaining data consistency.

Q72. How would you scale this application? A: 1. Add authentication and authorization

1. Migrate from SQLite to PostgreSQL
2. Add pagination for large datasets
3. Implement proper state management (Redux/Zustand)
4. Add comprehensive testing (Jest, React Testing Library)
5. Containerize with Docker

Extras

Q73. What is the purpose of the `key` prop in React lists? A: The `key` prop helps React identify which list items changed, were added, or removed. Without keys, React may re-render the entire list, causing performance issues and bugs.

Q74. What is the virtual DOM? A: React maintains a lightweight copy of the actual DOM in memory. When state changes, React updates the virtual DOM first, compares it with the previous version (diffing), and only updates the actual DOM where changes occurred.

Q75. What is hot module replacement (HMR)? A: A Vite feature that updates modules in the running application without a full page reload. When you edit a component, the change appears instantly while preserving the current application state.

Q76. What is the difference between npm and npx? A: `npm` installs packages from npm registry. `npx` executes packages without installing them globally. We use `npx prisma` to run the Prisma CLI without globally installing it.

Q77. What is a package.json file? A: A manifest file for Node.js projects. It lists project metadata, dependencies, and scripts. Our `package.json` defines 8 scripts (dev, server, build, etc.) and all package dependencies.

Q78. What is Vite? A: A modern build tool for web projects. It provides instant server start, lightning-fast hot module replacement, and optimized production builds. Uses Rollup under the hood for production bundling.

Q79. What is a hook in React? A: Hooks are functions that let you use React features (state, lifecycle, context) in functional components. Examples: `useState` (adds state), `useEffect` (adds side effects).

Q80. What is the difference between controlled and uncontrolled components? A: Controlled: React state controls the input value (`<input value={x} onChange={...} />`). Uncontrolled: DOM controls the value (using refs). We use controlled components for form validation.

Q81. What is a promise in JavaScript? A: A Promise represents a value that may be available now, later, or never. It has three states: pending, fulfilled, or rejected. Used for async operations like API calls.

Q82. What is the event loop? A: JavaScript's mechanism for handling async operations. It monitors the call stack and callback queue, executing queued callbacks when the stack is empty.

Q83. What is JSON? A: JavaScript Object Notation — a lightweight data interchange format. All API requests and responses use JSON: `{ "firstName": "Rajesh", "email": "rajesh@example.com" }`.

Q84. What is CORS? (See Q50)

Q85. What is REST? (See Q42)

Q86. What is a singleton pattern in Prisma? A: Prisma Client is designed to be a singleton — one instance per application. Creating multiple instances can cause connection pool exhaustion. Our server creates one `new PrismaClient()` at the top level.

Q87. What is the difference between PUT and PATCH? A: PUT replaces an entire resource. PATCH partially updates a resource. We use PUT for full updates (edit contact form) and PATCH for partial updates (just changing deal stage).

Q88. What is a foreign key constraint violation? A: An error that occurs when you try to create a record with a foreign key that doesn't exist in the referenced table. For example, creating a deal with a `stageId` that doesn't exist.

Q89. What is Prisma's upsert? A: An operation that updates a record if it exists, or creates it if it doesn't. Used in simulated lead sources: `prisma.contact.upsert({ where: { email }, update: {}, create: {...} })`.

Q90. What is SQLite good for? A: Development environments, testing, embedded applications, small projects. It requires zero configuration — just a file. Not ideal for high-concurrency production workloads.

Q91. What is the purpose of the seed file? A: To populate the database with realistic sample data so the application can be demonstrated immediately without manual data entry.

Q92. How many contacts, leads, and deals are seeded? A: 8 contacts, 8 leads, 7 deals, 1 pipeline with 7 stages, and 7 activities.

Q93. What is a KPI? A: Key Performance Indicator — a measurable value that demonstrates how effectively objectives are being achieved. In our dashboard: total contacts, leads, MQLs, SQLs, pipeline value, conversion rate.

Q94. What is Recharts? A: A composable charting library built on D3. Used for our dashboard charts: LineChart (lead acquisition), PieChart (source distribution), BarChart (status distribution).

Q95. What is the purpose of react-hot-toast? A: Provides toast notifications for user feedback. Used for success/error messages when creating leads, simulating sources, etc.

Q96. What is responsive design? A: Designing web pages that adapt to different screen sizes. In our app: sidebar collapses, grids reflow from 4 columns to 1, tables become scrollable.

Q97. What are webhooks? A: HTTP callbacks triggered by events. Not implemented, but would be used for real lead source integrations (e.g., Google Ads webhook fires when someone submits a form).

Q98. What is the difference between SQL and NoSQL? A: SQL databases (like SQLite) use structured tables with predefined schemas and relationships. NoSQL databases (like MongoDB) use flexible, document-based storage. We chose SQL for its relational nature (contacts → leads → deals).

Q99. What is an API client? A: A centralized module (`src/services/api.js`) that wraps all API calls. It handles base URL, JSON serialization, and error formatting. Components import from this file rather than making raw fetch calls.

Q100. What would you improve if you had more time? A: Authentication, drag-and-drop pipeline, email integration, pagination, testing suite, better error boundaries, mobile optimization, dark mode, real-time updates with WebSockets.

Q101. Is Nexus CRM production-ready? A: No. It lacks authentication, proper error boundaries, comprehensive testing, pagination, and production-grade infrastructure. It's a solid semester MVP that demonstrates full-stack development skills.

Q102. What did you learn from this project? A: Full React component architecture, state management with hooks, REST API design, database modeling with Prisma, responsive CSS with Tailwind, data visualization with Recharts, and end-to-end debugging.